

Aula 2

Design de Mídias Digitais

PROGRAMAÇÃO ORIENTADA A OBJETOS

Gustavo Fortunato Puga

Programação Orientada a Objetos

- Concentra as responsabilidades nos pontos certos
 - Encapsula a lógica de negócios

Go back to



O foco da plataforma

Aplicações de médio a grande porte, em que o time de desenvolvedores tem várias pessoas e sempre pode vir a mudar e crescer



Criar projetos, mesmo com o auxílio de IDEs e ferramentas poderosas, será mais trabalhoso que muitas linguagens script ou de alta produtividade. Porém, seguindo as boas práticas e recomendações sobre design orientado a objetos, será mais fácil e rápido fazer alterações no sistema desde que você.

O foco da plataforma

Aplicações de médio a grande porte, em que o time de desenvolvedores tem várias pessoas e sempre pode vir a mudar e crescer



Criar projetos, mesmo com o auxílio de IDEs e ferramentas poderosas, será mais trabalhoso que muitas linguagens script ou de alta produtividade. Porém, seguindo as boas práticas e recomendações sobre design orientado a objetos, será mais fácil e rápido fazer alterações no sistema desde que você.

Meu primeiro programa

Crie um arquivo chamado `MeuPrimeiroPrograma.java`

Nossa aplicação irá imprimir um texto simples

Meu primeiro programa

O Java é **CASE SENSITIVE**

Cuidado com maiúsculas e minúsculas.

```
1 public class MeuPrimeiroPrograma {  
2     public static void main(String[] args) {  
3         System.out.println("Estou vivo!!!");  
4     }  
5 }
```

A linguagem é bastante burocrática

Meu primeiro programa

Toda aplicação Java começa por um ponto de entrada: o método **main**

```
1 public class MeuPrimeiroPrograma {  
2     public static void main(String[] args) {  
3         System.out.println("Estou vivo!!!");  
4     }  
5 }
```

Meu primeiro programa

Abra o terminal e digite:

```
javac MeuPrimeiroPrograma.java
```

Será gerado um arquivo `MeuPrimeiroPrograma.class` (bytecode)

Meu primeiro programa

Visuale o arquivo do bytecode

```
javap -c MeuPrimeiroPrograma
```

Ofuscador

- Não impossibilita a engenharia reversa mas é difícil
- Torna o código fonte gerado pela engenharia reversa “menos” útil para hackers e concorrentes
- Não impede um atacante de desvendar como seu software funciona ou copiar seu código binário.

Nome do Arquivo Java

Experimente alterar o nome do arquivo para:

MinhaClasseMarota.java

Nome do Arquivo Java

```
> javac MinhaClasseMarota.java
```

MinhaClasseMarota.java:5: error:

class `MeuPrimeiroPrograma` is public, should be declared in a file
named `MeuPrimeiroPrograma.java`

```
public class MeuPrimeiroPrograma {
```

```
    ^
```

1 error

Regras para criar e nomear arquivos

- Não é necessário ter uma classe declarada com o mesmo nome do arquivo.
- Cada arquivo .java deve conter apenas uma classe pública.
- Caso uma classe tenha o modificador de acesso public, ela deve ter o mesmo nome do arquivo .java.
- Podemos ter mais de uma classe declarada em um mesmo arquivo.

Para testar



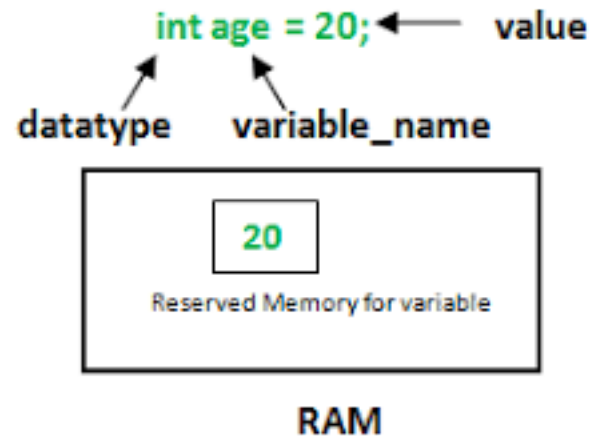
Experimente:

- Remover o modificador public da classe (compile e rode)

```
1 class MeuPrimeiroPrograma {  
2     public static void main(String[] args) {  
3         System.out.println("Estou vivo!!!");  
4     }  
5 }
```

Variáveis

Endereços de memória que tem um espaço ou tamanho definido de acordo com o tipo de dado que será guardado



Variáveis em Java

FORTEMENTE TIPADA

Toda variável tem um tipo que não pode ser alterado uma vez que é declarado

tipoDaVariavel nomeDaVariavel

Tipos de Variáveis em Java

2 tipos de dados divididos por: **valor** e **referência**.

- **Variáveis primitivas:** boolean, byte, char, short, int, long, float e double
- **Variáveis de referência:** Tipos de Objetos
 - Strings,
 - Arrays Primitivos,
 - Objetos

Variáveis primitivas

Tipos de informação mais usuais e básicos.

- Boolean: Não é um valor numérico
 - Só admite os valores: `true` ou `false`
- Char: Usa o código UNICODE e ocupa cada caractere 16 bits
 - Guarda apenas um caracter
 - Deve estar entre aspas simples: `'a'`
 - Vazio não é um caractere
 - Não guarda um código como `''`

Variáveis primitivas

- Inteiros: diferem nas precisões e podem ser positivos ou negativos
 - Byte: 1 byte
 - Short: 2 bytes
 - Int: 4 bytes
 - Long: 8 bytes
- Ponto flutuante: diferem nas precisões e podem ser positivos ou negativos
 - Float: 4 bytes.
 - Double: 8 bytes.

Variáveis primitivas

TIPO	TAMANHO
boolean	1 bit
byte	1 byte
short	2 bytes
char	2 bytes
int	4 bytes
float	4 bytes
long	8 bytes
double	8 bytes

Variáveis primitivas

O valor de um tipo primitivo sempre é **copiado**

```
int i = 5; // i recebe uma cópia do valor 5;  
int j = i; // j recebe uma cópia do valor de i;  
i = i + 1; // i vira 6, j continua 5.
```

Casting

MEIO PELO QUAL UM VALOR É "**TRANSFORMADO**" DE UM TIPO
PARA OUTRO

Quando existe a necessidade de arredondar números de ponto
flutuante para armazená-los em números inteiros, é preciso
"ORDENAR" que o número quebrado seja **moldado (casted)** como um
número inteiro para fazer que não ocorra erro de compilação.

Casting

PARA:	byte	short	char	int	long	float	double
DE:							
byte	----	<i>Impl.</i>	(char)	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>
short	(byte)	----	(char)	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>
char	(byte)	(short)	----	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>
int	(byte)	(short)	(char)	----	<i>Impl.</i>	<i>Impl.</i>	<i>Impl.</i>
long	(byte)	(short)	(char)	(int)	----	<i>Impl.</i>	<i>Impl.</i>
float	(byte)	(short)	(char)	(int)	(long)	----	<i>Impl.</i>
double	(byte)	(short)	(char)	(int)	(long)	(float)	----

Promoção

1. Se dois valores tiverem tipos de dados diferentes, será promovido automaticamente o menor tipo de dados para o maior.
2. Se um dos valores é inteiro e o outro é ponto flutuante, automaticamente será promovido o valor inteiro para o tipo de dados do valor de ponto flutuante.
3. Os tipos de dados menores, ou seja, *byte*, *short* e *char*, são promovidos primeiramente a qualquer momento para *int* no momento em que forem usados com um operador aritmético binário, mesmo que nenhum dos operandos seja *int*.
4. Após toda a promoção ter ocorrido e os operandos tiverem o mesmo tipo de dados, o valor resultante terá o mesmo tipo de dados da última promoção.

Discussão



Qual o tipo de *result*?

```
public class Promocao {  
    public static void main(String[] args) {  
        byte a = 10;  
        short b = 21;  
        int c = 32;  
        long d = 43L;  
        float e = 54.5F;  
        double f = 65.4D;  
        double result = (a * b / c * d / e * f);  
        System.out.println(result);  
    }  
}
```

Resposta



double

result = 309.5999959945679

Exercício



Fazer um programa em "Java" que receba o valor em graus Fahrenheit e imprime no vídeo o correspondente em graus Celsius usando as fórmulas:

- a) Usar uma variável *double* para ler o valor em Fahrenheit e a fórmula

$$C = (f - 32.0) * (5.0 / 9.0)$$

- b) Usar uma variável *int* para ler o valor em Fahrenheit e a fórmula

$$C = (f - 32) * (5 / 9)$$

Fazendo uma pausa nas variáveis



Antes de entrar na orientação a objetos vamos
aprender as estruturas de controle no Java

Estruturas condicionais: if ... else

IF é a estrutura condicional que executa a afirmação, dentro do bloco, se determinada condição for **verdadeira**. Se for **falsa**, executa as afirmações dentro de **ELSE**.

```
if (condicaoBooleana) {  
    codigo;  
}
```

Condição booleana é qualquer expressão que retorne true ou false

Condicionais aceitos

- MENOR QUE: $a < b$
- MENOR OU IGUAL: $a \leq b$
- MAIOR QUE: $a > b$
- MAIOR OU IGUAL: $a \geq b$
- IGUAL: $a == b$
- DIFERENTE DE: $a != b$

Estruturas condicionais: if ... else

Complementar ao `if/else` temos o operador `else if`, que possibilita adicionar uma nova condição à estrutura de decisão para atender a lógica sendo implementada.

```
if (condicaoBooleana1) {  
    // bloco de código 1  
} else if (condicaoBooleana2) {  
    // bloco de código 2  
} else {  
    // bloco de código 3  
}
```

Operador ternário

Similar ao do `if/else`, mas que é codificado em apenas uma linha.

```
(condiçãoBooleana) ? código 1 : código 2;
```

Ao avaliar a expressão booleana, caso ela seja verdadeira, o código 1, declarado após o ponto de interrogação (?) será executado; do contrário, o programa irá executar o código 2, declarado após os dois pontos (:).

Estruturas condicionais: switch

Possibilita diversos desvios, ou seja, de acordo com o resultado de uma condição pode-se executar um desvio entre os vários possíveis.

Estruturas condicionais: switch

```
char operacao = '+';  
switch (operacao) {  
    case '+':  
        //Instruções;  
        break;  
  
    case '-':  
        //Instruções;  
        break;  
  
    case '*':  
        //Instruções;  
        break;  
  
    default  
        //Instruções;  
        break;  
}
```

Tipos de valores aceitos: **char**, **byte**, **int**, **short** ou um
tipo **enumerado**

Tipos enumerados não serão abordados agora.

Laços de repetição: while

Usado **quando não se sabe quantas vezes** um determinado bloco de instruções **precisa ser repetido**

A execução das instruções continua até que uma condição seja verdadeira. A **condição** a ser analisada para a execução do laço de repetição **deverá retornar um valor booleano**.

Laços de repetição: while

Se a primeira verificação da condição for falsa, o programa simplesmente irá "pular" para a execução da próxima instrução após o laço

```
while (teste condicional){
```

```
//comandos; > serão executados enquanto o teste condicional for igual a verdadeiro
```

```
}
```

Laços de repetição: do ... while

Semelhante ao while, porém os comandos dentro do bloco serão
executados ao menos uma vez

```
do{  
  
    //comandos;  
  
} while (condicao);
```

Laços de repetição: for

Usado **quando se sabe** quantas vezes é necessário
repetir um bloco de instruções

```
for (valor inicial; teste booleano; incremento){  
  
    //Lógica  
  
}
```

Laços de repetição: for

Na primeira parte da instrução for, podemos declarar e inicializar uma ou mais variáveis (separadas por vírgula)

```
for (int a = 0, b = 1, c = 2; a < 5; a++, b++, c++) {  
  
}
```

A inicialização acontece sempre antes dos outros comandos e apenas uma vez no laço de repetição

Para testar

Experimente: Esse código funciona?

```
public class TesteFor {  
    public static void main(String[] args) {  
        boolean condicao = false;  
        for (; !condicao;) {  
            System.out.println(condicao);  
            condicao = true;  
            System.out.println(condicao);  
        }  
    }  
}
```

Controle de loops: break e continue

Apesar das condições booleanas nos laços, em algum momento, pode ser necessário parar o loop por algum motivo especial sem que o resto do laço seja executado.

Controle de loops: break

Usado em laços de repetição `while`, `do/while`, `for` e com os comandos `switch/case`.

```
public class TesteBreak {  
  
    public static void main(String[] args) {  
  
        for (int i = 1; i < 10; i++) {  
            if (i == 5) {  
                break;  
            }  
            System.out.println("Valor de i = " + i);  
        }  
  
    }  
  
}
```

Quando usado em laço de repetição, causa uma interrupção imediata do mesmo, continuando a execução do programa na próxima linha após o laço. Isso ocorre caso a condição imposta seja atendida.

Controle de loops: continue

Usado **somente em laços de repetição**. Quando executado, o laço volta imediatamente para o teste de condição do laço de repetição.

```
public class TesteContinue {  
  
    public static void main(String[] args) {  
  
        int i = 1;  
  
        while (i < 5){  
            if (i == 4){  
                continue;  
            }  
            System.out.println("Valor de i = " + i);  
        }  
  
    }  
  
}
```

Normalmente usado em conjunto com o condicional if.

Escopo de variáveis

NOME DADO AO TRECHO DE CÓDIGO EM QUE AQUELA VARIÁVEL
EXISTE E O LUGAR ONDE É POSSÍVEL ACESSÁ-LA

É possível declarar variáveis a qualquer momento

As variáveis declaradas num bloco só valem dentro dele.

Escopo de variáveis

```
// aqui, a variável i não existe.  
int i = 5;  
// a partir daqui, ela existe.  
while (condicao) {  
    // o i ainda vale aqui.  
    int j = 7;  
    // o j passa a existir.  
}  
// aqui, o j não existe mais, porém o i continua dentro do escopo.
```

Exercício



A Sequência de Fibonacci tem como primeiros termos os números 0 e 1 e, a seguir, cada termo subsequente é obtido pela soma dos dois termos predecessores. Nenhuma outra sequência de números foi tão estudada e possui aplicações em áreas tão distintas, como por exemplo Biologia, Arquitetura, Arte e outras. A razão entre dois números consecutivos da sequência converge para um valor constante de 1,618... conhecido como número de ouro.

- a) Imprima os 30 primeiros números da sequência de Fibonacci
- b) Imprima os primeiros números da série de Fibonacci menores que 100

Regra:

$$F_n = F_{n-1} + F_{n-2}$$

Próxima aula

Para aprender mais:

 <https://loiane.training/curso/java-basico>

 <https://www.caelum.com.br/apostila-java-orientacao-objetos/>

 <https://www.alura.com.br/artigos/poo-programacao-orientada-a-objetos>

 Atributos e métodos

 Java

Bibliografia

BARNES, David J.; KOLLING, Michael. **Programação orientada a objetos com Java**. 4. ed. São Paulo: Prentice Hall – Br, 2009.

BOENTE, Alfredo. **Aprendendo a programar em C++: usando classes e objetos**. Rio de Janeiro: Brasport, 2004. ISBN 9788574521749.

CORREIA, Carlos Henrique; TAFNER, Malcon Anderson. **Análise orientada a objetos**. 2. ed. Florianópolis: Visual Books, 2006.

DEITEL, Paul; DEITEL, Harvey. **Java: como programar**. 10. ed. São Paulo: Pearson, 2017. ISBN 9788543004792.

Sierra, Kathy. **Use A Cabeça Java**. S.l: Alta Books, 2009. ISBN: 8576081733



Obrigado!

Gustavo Fortunato Puga

gustavo.puga@fatec.sp.gov.br